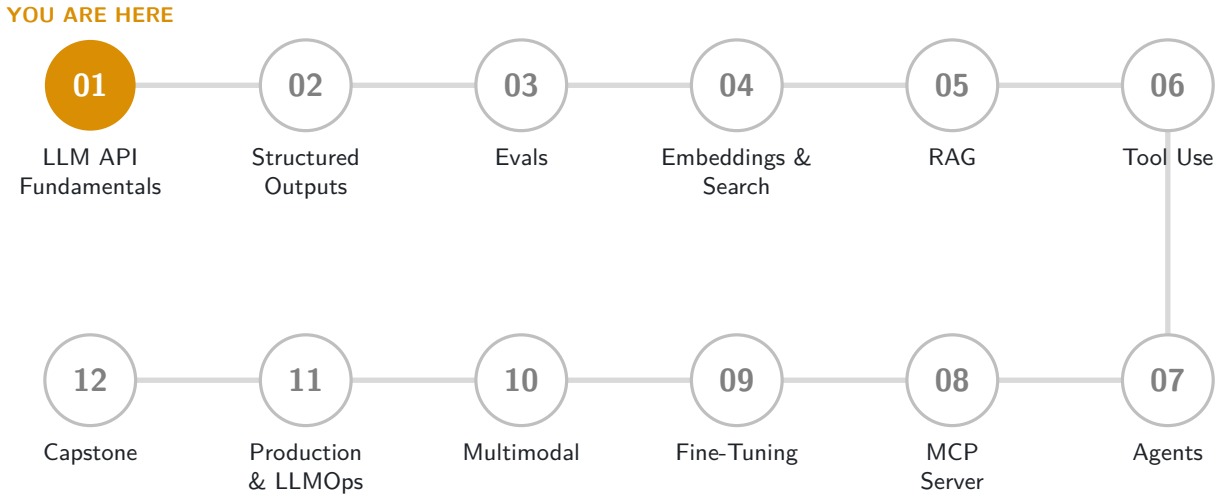
AI ENGINEERING

Field Notes · a project-based quest log



KEY IDEA	the concept to remember
RULE	how to do it right
TRAP CARD	a mistake that actually happened
BOSS FIGHT	the hard part of a level
LAB RESULT	numbers measured in our own runs
ACHIEVEMENT	what you can now do

Contents

1 LLM API Fundamentals **2**

1.1 Step 1: The first call . . . 2

Anatomy of a request . . . 2

Anatomy of a response . . . 3

Tokens are money . . . 3

The `max_tokens` boss . . . 3

`stop_reason`: status effects . . . 4

1.2 Step 2: A chat that remembers . . . 4

How the cost grows . . . 5

Sessions . . . 5

1.3 Types: dicts in, objects out . . . 5

1.4 API vs SDK . . . 6

1.5 Step 3: Streaming . . . 7

One stream = one reply . . . 7

TTFT vs total time . . . 7

Streaming inside the chat: two loops . . . 8

Why streaming matters beyond looks . . . 8

Level 01 progress . . . 8

LEVEL 01

LLM API Fundamentals

Mission: build a command-line chat assistant on the raw API, no frameworks, and understand exactly what every call costs and returns.

Step	Quest	Status	XP
1	The first call	CLEARED	50
2	A chat that remembers	CLEARED	65
3	Streaming	CLEARED	60
4	Settings: system prompt, model, effort	NEXT	–
5	Token budget: trim or summarize	LOCKED	–
6	Errors and retries	LOCKED	–

EQUIPPED MODEL

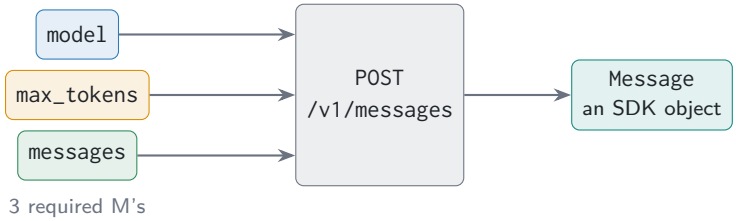
`claude-haiku-5-5` $0.10 per 1M input tokens · $0.50 per 1M output tokens (prompts up to 100K tokens; above that $0.50 / $2.50).
Step 1 was first run on `claude-opus-5-5` ($4 / $20). We switched because a learning project does not need the biggest model.

1.1 Step 1: The first call

+50 XP

Anatomy of a request

Every call to a model is one HTTPS request to one endpoint. It has three required parts.



3 required M's

```
response = client.messages.create(
    model="claude-haiku-5-5",    # which model answers
    max_tokens=4000,             # hard cap on the reply (thinking + text)
    messages=[{"role": "user", "content": "Explain what a token is in 2 sentences"}],
)
```

★ **KEY IDEA: the model is stateless**

Each call starts from zero. The model remembers nothing between calls. Anything it should know (the whole conversation so far) has to be inside *this* request.

Anatomy of a response

Field	Type	Meaning
`.content`	`list[ContentBlock]`	the reply, as a list of blocks: `thinking`, then `text`
`.stop_reason`	`str`	why the model stopped generating
`.usage.input_tokens`	`int`	everything you sent: system prompt + full history + overhead
`.usage.output_tokens`	`int`	everything generated, **including hidden thinking**

```python
for block in response.content:
    if block.type == "text":        # never assume content[0] is the text
        print(block.text)
```

⚠ TRAP CARD

`content[0]` is not always the text. Opus 5.5 returned a `thinking` block first; Haiku 5.5 skipped thinking on the same easy question and returned only `text`. Current models decide for themselves how much to think, so loop and check `block.type`.

Tokens are money

A **token** is the unit a model reads, writes and bills: roughly $\frac{3}{4}$ of a word. *unbelievable* might become `un` `believ` `able`. An 8-word prompt cost 22 input tokens, because the message wrapper adds overhead.

$$\text{cost} = \frac{\text{input tokens}}{10^6} \times p_{\text{in}} + \frac{\text{output tokens}}{10^6} \times p_{\text{out}}$$

Output tokens cost $5\times$ input tokens on both models, and thinking counts as output.

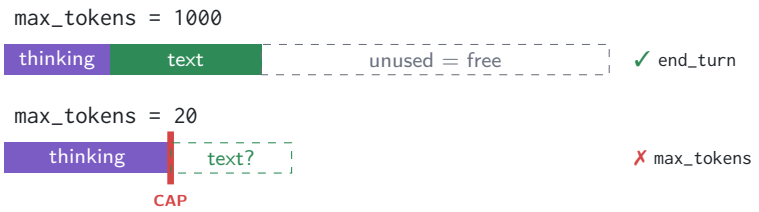
▲ LAB RESULT measured, not guessed

Same question (*Explain what a token is in 2 sentences*), one run each:

Model	in	out	cost
`claude-opus-5-5`	22	142	$0.002928
`claude-haiku-5-5`	22	73	$0.000039

Haiku was $75\times$ cheaper: $40\times$ from the price, the rest because it skipped thinking. A second Opus run of the same question used 168 output tokens instead of 142: **same prompt ≠ same output**, so measure more than one run.

The `max_tokens` boss



■ BOSS FIGHT: thinking ate the whole budget

With `max_tokens=20` on Opus, `stop_reason` was `max_tokens` and there was **no text at all**: thinking used all 20 tokens before the answer started. We were still billed for 20 output tokens.

Strategy: `max_tokens` is a ceiling, not a target, and unused tokens are free. Set it generously (4000) and always check `stop_reason`.

stop_reason: status effects

`end_turn`	finished normally: safe to use
`max_tokens`	hit your cap: the answer is incomplete (or missing)
`tool_use`	wants *you* to run a tool (Level 06)
`refusal`	the model declined the request
`model_context_window_exceeded`	the history no longer fits the model (Step 5)

➤ RULE

Check `stop_reason` **before** trusting `content`. It is a plain `str`, so compare with `==`.

⚠ TRAP CARD

API keys never go in code. Ours lives in a Windows user environment variable; a `.env` file (listed in `.gitignore`) also works. `anthropic.Anthropic()` finds it by itself.

★ ACHIEVEMENT UNLOCKED: First Call · Priced a Call · Beat the Boss

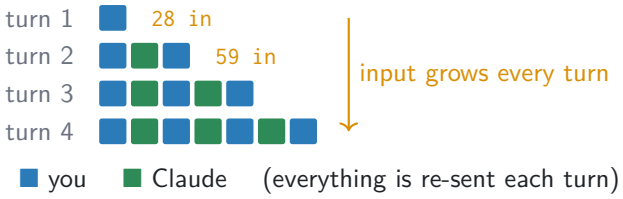
You can make a call, read every part of the response, and say what it cost.

1.2 Step 2: A chat that remembers

+65 XP

★ KEY IDEA: memory is a list you resend

A chat app fakes memory. It keeps the whole conversation in a list and sends **all of it** with every call. Forget to append the assistant's reply and the model never sees what it said before.




```python
messages = []                                   # the whole session lives here
while True:
    user_text = input("You: ").strip()
    if user_text.lower() in ["quit", "exit"]:
        break
    messages.append({"role": "user", "content": user_text})
    response = client.messages.create(model=MODEL, max_tokens=MAX_TOKENS, messages=messages)
    messages.append({"role": "assistant", "content": response.content})   # ALL blocks
```

➤ RULE

Append `response.content` (all blocks, thinking included), not just the text. The API expects thinking blocks to come back unchanged.

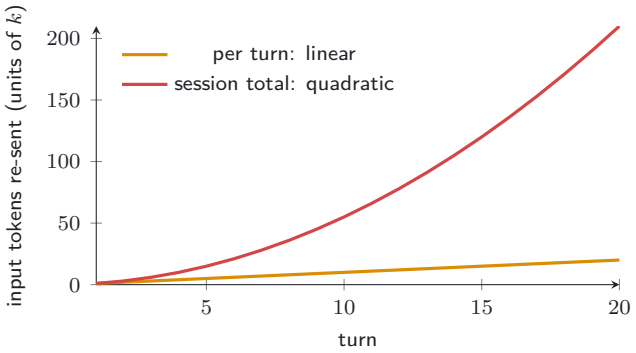
▲ LAB RESULT measured, not guessed

Turn 1: *My name is Sam* → 28 input tokens. Turn 2: *What is my name?* → 59 input tokens, and it answered *Sam*. Sending only the latest message instead of the list: it forgot the name, and input tokens dropped back down.

How the cost grows

If each exchange adds about k tokens, turn n re-sends about $n \cdot k$ tokens. So:

$$\text{cost of turn } n \propto n \quad \text{(linear)} \qquad \text{session total after } n \text{ turns} \propto \frac{n(n+1)}{2} \quad \text{(quadratic)}$$



> ⚠ **TRAP CARD**
> The first thing that breaks is not the context window (1M tokens on Haiku 5.5) but the **price**. Once one request is over 100K tokens, Haiku charges $0.50 / $2.50 per 1M instead of $0.10 / $0.50. Eventually `stop_reason` becomes `model_context_window_exceeded`. Step 5 fixes this by trimming or summarizing old history.

Sessions

1 run = 1 session = 1 list. When the script exits, the list is gone; the API stores nothing. Apps like Claude.ai keep each conversation in a database and resend it. To make a session survive a restart, save the list to JSON (see the next section for why `model_dump()` is needed).

> ⚠ **TRAP CARD**
> `"\$"` inside a Python string is not an escape: it prints a literal backslash and raises a `SyntaxWarning`. Just write `$`.

> ★ **ACHIEVEMENT UNLOCKED: Gave Claude a Memory**
> You can build a multi-turn chat and explain why long chats get expensive.

1.3 Types: dicts in, objects out

> ★ **KEY IDEA**
> **You send plain dicts. You get SDK objects back.** Mixing them up causes most beginner errors.

YOU SEND `dict`

```
messages = [
  {"role": "user",
   "content": "My name is Sam"},
  {"role": "assistant",
   "content": response.content},
]
```

Read with **brackets**: `msg["role"]`. `MessageParam` is only a type hint for your editor; at runtime it is a plain dict.

YOU GET `Message` (a pydantic object)

```
response: Message
├─ .content: list[ContentBlock]
│   ├─ ThinkingBlock .thinking
│   └─ TextBlock .text
├─ .stop_reason: str
└─ .usage: Usage
    ├─ .input_tokens: int
    └─ .output_tokens: int
```

Read with **dots**: `block.text`. `ContentBlock` is a union of block types, so check `block.type` first.

➤ **RULES OF THE GAME**

✓ `block.text`
objects are read with dots

✗ `block["text"]`
`TypeError: 'TextBlock' object is not subscriptable`

✓ `[b.model_dump() for b in response.content]`
turns objects into JSON-safe dicts (needed to save a session)

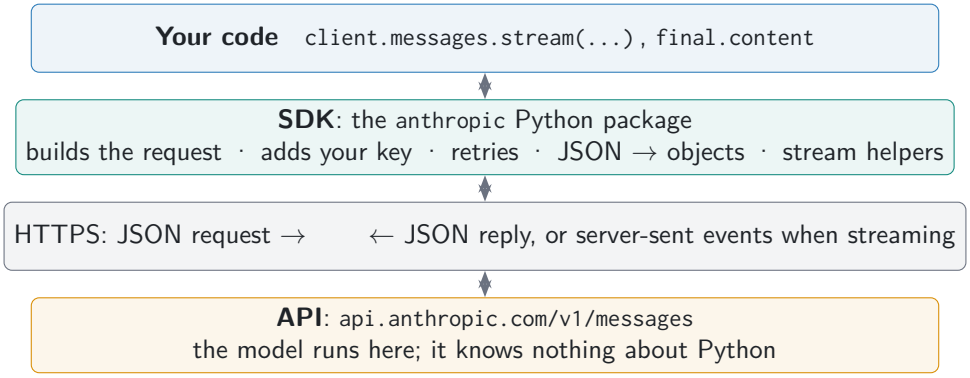
✗ `json.dumps(response.content)`
`TypeError: Object of type TextBlock is not JSON serializable`

✓ `{"role": "assistant", "content": response.content}`
the SDK accepts its own objects back, so no conversion is needed here

▲ **LAB RESULT** measured, not guessed

Both errors above were reproduced against the installed SDK (`anthropic` 2.0.0), not recalled from memory.

1.4 API vs SDK



➤ **RULE: the curl test**

Could I do this with `curl`*?* **Yes** → it is the API: field names (`max_tokens`, `stop_reason`, `usage`) and stream event names. **No** → it is the SDK: Python methods and classes such as `get_final_message()`, `text_stream`, `TextBlock`, `model_dump()`.

▲ **LAB RESULT** measured, not guessed

A raw streaming call with `curl` and no Python (shortened):

```
event: message_start
data: {"type":"message_start","message":{... "usage":{"input_tokens":17, ...}}}
event: content_block_start
data: {"type":"content_block_start","index":0,"content_block":{"type":"text","text":""}}
event: ping
event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":"Hello there,"}}
event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":" friend!"}}
event: content_block_stop
event: message_delta
data: {"type":"message_delta","delta":{"stop_reason":"end_turn"},"usage":{"output_tokens":10, ...}}
event: message_stop
```

No `text`, `thinking` or `signature` events: those are added by the SDK.

1.5 Step 3: Streaming

+60 XP

★ **KEY IDEA: delivery, not content**

Streaming changes **how** the answer arrives, not **what** it is: same model, same answer quality, same tokens, same cost. The connection stays open and the reply arrives in pieces (events) while it is being written.

One stream = one reply

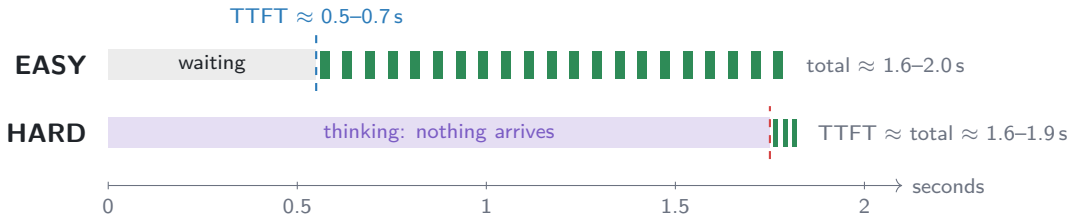
Event	How often	Carries
`message_start`	once	message id, model, starting usage
`content_block_start`	once per block	which block begins: `thinking` or `text`
`content_block_delta`	many	the pieces: `thinking_delta`, `text_delta`, `signature_delta`
`content_block_stop`	once per block	block finished
`message_delta`	once	`stop_reason` + final usage
`message_stop`	once	reply finished

⚠ **TRAP CARD**

The Python SDK adds extra events named `text`, `thinking` and `signature` that *repeat* the delta data. Printing both doubles your output. (The raw API also sends `ping` keep-alives.)

TTFT vs total time

Metric	Meaning	Streaming helps?
TTFT (time to first token)	how long the user stares at nothing	✓ a lot
Total time	time until the last word	✗ about the same



▲ **LAB RESULT** measured, not guessed

Haiku 5.5, two runs each. EASY (*why is the sky blue*) did not think: first text after 0.47–0.67 s, done after 1.61–1.97 s. HARD (a marbles puzzle) thought first: nothing arrived for 1.63–1.94 s, then the thinking summary and the answer came in one burst. A hard question can triple TTFT.

Streaming inside the chat: two loops

```
while True:                                          # OUTER loop: one pass per turn
    ...
    messages.append({"role": "user", "content": user_text})
    print("\nClaude: ", end="", flush=True)
    with client.messages.stream(model=MODEL, max_tokens=MAX_TOKENS,
                                messages=messages) as stream:
        for text in stream.text_stream:              # INNER loop: one pass per piece
            print(text, end="", flush=True)
        final = stream.get_final_message()           # the same Message create() returns
    print()                                          # streamed text never ends in a newline
    messages.append({"role": "assistant", "content": final.content})
```

▲ **LAB RESULT** measured, not guessed

Same question through the step 2 chat and the streaming chat: both `29 in / 11 out, $0.000008`. Streaming did not change the cost.

⚠ **TRAP CARD: the silent pause**

On a hard question there is a pause after `Claude:`. The model is thinking, and two things hide it: `text_stream` only yields `text_delta` pieces, and thinking text is empty by default anyway (it only has text with `thinking={"type": "adaptive", "display": "summarized"}`). Those thinking tokens are still billed as output.

Why streaming matters beyond looks

- **Long requests stay alive.** A non-streaming call sits silent until the end and can time out; the SDK refuses non-streaming requests it estimates will take more than about 10 minutes. A stream keeps sending data (`ping`).
- **You can act early:** a stop button, cancelling halfway, processing the start of the answer.
- **Partial failure is possible:** a dropped connection leaves half an answer. `get_final_message()` assembles the full `Message` for you.

★ **ACHIEVEMENT UNLOCKED: Read the Stream · Measured TTFT**

You can explain every event in a stream, measure what the user actually waits for, and stream a multi-turn chat.

Level 01 progress

S1 First call	S2 Memory	S3 Streaming	S4 Effort	S5 Budget	S6 Retries

175 XP earned **Next quest:** Step 4, settings and *effort* (the replacement for temperature).